

cuTile 기반 Windows-native LLM 추론 엔진 이식 및 코딩 에이전트 협력 설계 실증 연구

(Windows-Native LLM Inference Engine Migration via cuTile and Human-Agent Co-Design: An Empirical Study)

김태원
(Taewon Kim)

요약

본 연구는 Linux 중심 LLM 서버 프레임워크의 Windows 환경 이식성 제약을 완화하기 위해 cuTile과 CUDA Python 1.0 기반 Windows-native LLM 추론 엔진 **micro-vllm**을 설계하고 구현하였다. **micro-vllm**은 **flash_attn** 및 **nano-vllm**의 핵심 커널을 **MatMul**, **FMHA**, **llm-from-scratch** 단계로 마이그레이션하며, 실험 설계와 실패 **insight**를 DSL로 누적하는 코딩 에이전트 협력 프레임워크를 함께 제시한다. WSL2 FlashAttention은 평균 2138.55 tok/s로 현재 cuTile backend보다 약 4.62 배 높은 처리량을 보였으며, Green Contexts 기반 Prefill/Decode SM 분할은 Decode P99 ITL을 평균 3.9% 낮추었지만 P50 ITL과 처리량은 각각 5.2%, 2.6% 악화되었다. 또한 shape padding 기반 launch/JIT 회피 시도가 PyTorch caching allocator 스레싱을 유발하여 전체 생성 처리량을 67.0% 저하시킬 수 있음을 관찰하였다.

핵심어: 거대 언어 모델(LLM) 서버, cuTile, CUDA Python 1.0, 단계적 커널 마이그레이션, 코딩 에이전트 공동 설계

Abstract To alleviate the portability constraints of Linux-centric LLM serving frameworks in Windows environments, this study designs and implements **micro-vllm**, a Windows-native LLM inference engine based on cuTile and CUDA Python 1.0. **micro-vllm** migrates core kernels from **flash_attn** and **nano-vllm** through sequential stages: **MatMul**, **FMHA**, and **llm-from-scratch**. Additionally, we present a human-agent co-design framework that accumulates experimental designs and failure insights using a Domain-Specific Language (DSL). Experimental results show that while WSL2 FlashAttention achieves an average throughput of 2138.55 tok/s—approximately 4.62 times higher than the current cuTile backend—the Green Contexts-based Prefill/Decode SM partitioning reduces the Decode P99 Inter-Token Latency (ITL) by an average of 3.9%, though it degrades the P50 ITL and overall throughput by 5.2% and 2.6%, respectively. Furthermore, we observe that attempts to avoid kernel launch and JIT compilation overhead via shape padding can trigger PyTorch caching allocator thrashing, leading to a 67.0% reduction in total generation throughput.

Keywords: LLM Serving, cuTile, CUDA Python 1.0, Stepwise Kernel Migration, Coding Agent Co-design

1. 서론

최근 생성형 AI 기술의 확산에 따라 로컬 온프레미스 장비나 오프라인 연구실 환경에서 LLM을 구동하려는 수요가 증가하고 있다. 특히 RAG(Retrieval-Augmented Generation), 로컬 지식베이스 구축, 그리고 기업 정보 보안을 고려한 로컬 프라이빗 추론 환경은 실제 시스템 설계에서 중요한 요구사항으로 자리 잡고 있다. 그러나 vLLM이나 TensorRT-LLM 등 실용적으로 널리 쓰이는 고성능 서버 아키텍처는 PagedAttention 기반 메모리 관리 [1], Triton 커널 컴파일러 [3], FlashAttention 패키지 [4], 분산 통신용 NCCL 라이브러리 등 Linux 중심 구성 요소에 의존하는 경우가 많다. 그 결과 데스크톱 및 로컬 서버 환경의 보편적 운영 체제인 Windows에서는 WSL2(Windows Subsystem for Linux)를 통한 우회 구동을 선택하는 경우가 많으며, 이 과정에서 파일 시스템 경계, 드라이버/런타임 구성 차이, Linux 전제 빌드 스크립트는 Windows native AI application 개발과 디버깅의 장애 요인으로 작용한다.

본 연구는 이러한 크로스 플랫폼 제약을 완화한 로컬 LLM 서

버 환경 구축을 목표로, Windows OS 기반의 **cuTile** Python DSL 및 **CUDA Python 1.0**을 활용한 서버 엔진 [2, 6] **micro-vllm**을 개발하였다. 본 아키텍처는 진입 장벽이 높은 C++ CUDA 대신 Python 수준에서 타일 기반 커널을 표현할 수 있는 cuTile을 도입하여 커널 코드 분석과 최적화 과정을 비교적 명시적으로 관찰할 수 있도록 하였으며, 이를 통해 추론 성능과 시스템 병목 분석에 집중할 수 있도록 구성하였다. 구현은 **MatMul**, **FMHA**, 초소형 **llm-from-scratch**, **nano-vllm** 커널 마이그레이션으로 이어지는 단계적 확장 경로를 포함한다. 이러한 커널 마이그레이션 접근은 AI 에이전트를 활용해 TensorFlow 모델을 JAX로 이전하는 migration workflow 연구와 마찬가지로, 기존 코드의 구조를 분석하고 목표 실행 환경에 맞는 변환 계획을 수립하는 문제와 연결된다 [8]. 이 과정에서 발생하는 튜닝 및 하드웨어 프로파일링의 긴 탐색 비용을 줄이기 위해, AI 코딩 에이전트와의 협력적 튜닝 파이프라인을 적용하는 실험과학적 엔지니어링 프로세스를 검토하였다. 이는 명세 기반 에이전트 워크플로에서 repository context와 검증 hook을 통해 에이전트의 추론을 환경에 고정하려는 최근 연구 흐름과도 맞닿아 있다 [9].

본 논문은 개별 커널을 빠르게 만드는 최적화가 전체 LLM 서빙 루프에서는 항상 성능 향상으로 이어지지 않는다는 점을 구체적인 사례로 분석한다. 예를 들어 kernel launch나 JIT compile 반복을 줄이기 위해 입력 shape을 고정하거나 padding을 추가하면, 컴파일 비용은 줄어든 수 있지만 매 decode step마다 임시 텐서 할당과 KV cache 갱신 비용이 증가할 수 있다. 본 연구는 이러한 비용이 PyTorch CUDA caching allocator와 호스트-디바이스 동기화 병목으로 이어지는 과정을 분석하였다. 또한 Prefill과 Decode가 같은 GPU 자원을 경쟁적으로 사용할 때 발생하는 간섭을 줄이기 위해 CUDA Green Contexts 기반 SM 분할 [5]을 적용하고, 그에 따른 디코딩 지연시간과 처리량 변화를 반복 측정 및 분석하였다. 추가로 WSL2 환경에서 FlashAttention backend를 구동하여, Windows native cuTile 경로와 성숙한 Linux 기반 attention stack 사이의 성능 차이를 기준선으로 확인하였다.

1.1 본 연구의 주요 기여

본 논문이 제시하는 학술적 및 시스템적 주요 기여점은 성능 상위 주장보다 재현 가능한 구현 산출물, 실험 절차, 그리고 병목 분석에 초점을 맞추어 다음과 같이 정리된다.

- **[Education & Kernel Migration] 단계적 커널 마이그레이션 기반 LLM 추론 엔진 학습 자료:** flash_attn 라이브러리 및 Linux 기반 nano-vllm의 핵심 커널을 cuTile 및 CUDA Python 1.0 기반으로 마이그레이션하여, 보편적 운영체제인 Windows 환경에서 동작하는 **micro-vllm**을 설계하고 실증적으로 구현했다. LLM inference engine 최적화에 처음 입문하는 연구자와 학습자가 시스템 구조를 단계적으로 이해할 수 있도록, micro-vllm은 MatMul, FMHA, 그리고 초소형 llm-from-scratch를 순차적으로 구현하고 각 단계에서 cuTile과 PyTorch의 성능을 비교하는 형태로 구성하였다. 이를 통해 파이썬 생태계 안에서 GPU 커널, 어텐션, KV 캐시, 서빙 루프가 연결되는 과정을 재현 가능한 학습 경로로 제시하였다.
- **[AI4Sys] 지식-주도형 코딩 에이전트 협력 프레임워크 (Human-AI Co-design Framework):** 본 연구는 커널 최적화 과정을 단발성 코드 생성이 아니라 DSL 기반의 통제된 과 학실험 루프로 구조화하였다. MatMul, FMHA, LLM-from-scratch, micro-vllm 단계에서 각 실험의 가설, 튜닝 파라미터, 하드웨어 제약, acceptance criteria, 실험 결과, 실패 원인, 후속 insight를 DSL 형식으로 누적하여 다음 실험 설계에 재 사용하였다. 특히 TechLead 에이전트는 실험을 직접 실행 하지 않고 실험 설계, 코드 패치, 로그 분석, 결과 해석만 담당하도록 제한하였다. 이 실험 권한의 분리는 TechLead가 로컬 디버깅의 우발적 세부사항에 매몰되지 않고, 주어진 시스템에 대한 멘탈 모델을 유지하면서 아키텍처 불변성, 물리

제약, 성능 인과관계를 일관되게 추론하도록 강제하는 역할을 했다.

- **[System & Methodology] 단일 GPU 내 Prefill/Decode 자원 격리 구현 및 평가 (Single-GPU Prefill/Decode Isolation with Green Contexts):** vLLM 계열 LLM serving에서는 대량 토큰을 한 번에 처리하는 Prefill 단계와 1토큰 단위로 반복되는 Decode 단계의 병목 특성이 다르기 때문에, 두 단계의 자원 간섭을 줄이는 분리 실행 구조가 중요하다. 본 연구는 CUDA Python 1.0의 Green Contexts API를 이용하여 단일 RTX 5070 GPU 안에서 SM 자원을 Prefill 32 SM, Decode 16 SM으로 물리적으로 나누는 방식을 **micro-vllm**에 적용하였다. 이를 통해 별도 GPU나 분산 런타임 없이 하나의 소비자용 GPU 내부에서 Prefill과 Decode의 실행 영역을 분리하고, Decode 작업의 L2 cache residency를 보호하는 설계를 검토하였다. 총 8회 반복 실행에서는 Decode P99 ITL이 평균 3.9% 감소했지만, P50 ITL과 전체 Throughput은 각각 5.2%, 2.6% 악화되었다. 따라서 본 연구는 Green Contexts를 확정적인 성능 향상 기법으로 단정하지 않고, 단일 GPU 내부에서 Prefill/Decode 자원 격리를 구현하고 tail latency와 throughput 사이의 trade-off를 검토하기 위한 실험적 기반으로 제시한다.
- **[Engineering Analysis] CUDA Graph/JIT 회피 최적화의 역효과 분석 (Allocator Thrashing under Shape Stabilization):** LLM serving에서 빈번한 kernel launch와 JIT compile 반복을 줄이기 위해 CUDA Graph 적용이나 shape padding을 통한 형상 안정화가 흔히 고려된다. 그러나 본 구현 범위의 Eager-mode micro-vllm에서는 이러한 접근이 예상과 달리 성능을 저하시킬 수 있음을 관찰하였다. 특히 동적 배치와 Paged KV cache가 결합된 상황에서 padding은 커널 컴파일 횟수를 줄이는 대신, 매 decode step마다 대형 임시 텐서 할당과 복사를 유발하였다. 그 결과 PyTorch caching allocator의 free-list 탐색, cudaMalloc/cudaFree fallback, Python GC 및 암묵적 동기화 비용이 누적되어 GPU kernel launch 또는 JIT compile 비용보다 큰 호스트-디바이스 병목으로 전환되었다. 실험에서는 padded 경로가 no-padding Eager 경로 대비 전체 Throughput을 67.0% 낮추는 것으로 관찰되었으며, 이를 통해 LLM inference engine에서는 launch/JIT 회피 전략도 메모리 할당 패턴과 KV cache 갱신 방식까지 함께 고려해 평가해야 함을 보였다.

2. 관련 연구 및 배경

2.1 LLM 서빙 엔진과 PagedAttention

LLM 서빙 엔진은 사용자의 입력 프롬프트를 한 번에 처리하는 Prefill 단계와, 이후 토큰을 하나씩 생성하는 Decode 단계를 반복적으로 수행한다. Decode 단계에서는 매 스텝마다 과거 토큰의 Key/Value 정보를 재사용하므로, KV cache의 배치, 갱신, 재사용 방식이 지연시간과 처리량에 직접적인 영향을 준다. vLLM은 KV cache를 고정 크기의 물리 블록으로 나누어 관리하는 PagedAttention을 통해 메모리 단편화와 동적 배치 문제를 완화하였다 [1]. 이 구조는 대규모 multi-user serving에서 효과적이지만, 실제 구현은 Triton [3], FlashAttention [4], NCCL 등 Linux 중심 GPU 소프트웨어 스택과 밀접하게 결합된 경우가 많다. 따라서 Windows 기반 로컬 장비나 교육용 단일 GPU 환경에서는 서빙 구조를 학습하거나 커널 수준에서 수정·분석하기 어렵다는 제약이 남는다. 본 연구의 micro-vllm은 이러한 제약을 줄이기 위해 Paged KV cache와 Prefill/Decode 실행 흐름을 Windows native 환경에서 재구성하는 것을 목표로 한다.

2.2 CUDA Python 1.0 및 cuTile DSL

CUDA Python 1.0은 Python에서 CUDA driver/runtime API를 직접 호출할 수 있는 바인딩 계층을 제공하며, Python 기반 실험 코드와 저수준 GPU 제어를 연결하는 기반이 된다. cuTile DSL은 행렬 곱셈이나 어텐션 연산을 타일 단위로 표현하여, shared memory, register blocking, Tensor Core MMA와 같은 커널 설계 요소를 Python 코드 수준에서 다룰 수 있도록 한다 [2, 6]. 이는 C++ CUDA보다 진입 장벽이 낮고, PyTorch의 고수준 연산보다 하드웨어 동작을 명시적으로 관찰하기 쉽다는 장점이 있다.

본 연구에서는 cuTile을 단순한 커널 작성 도구가 아니라 단계적 학습 및 마이그레이션 도구로 사용한다. 먼저 MatMul에서 타일링과 메모리 계층을 학습하고, FMHA에서 online softmax와 causal masking을 구현한 뒤, llm-from-scratch 단계에서 KV cache와 CUDA Graph를 결합한다. 마지막으로 nano-vllm의 핵심 커널을 micro-vllm으로 마이그레이션하여 Paged KV cache와 continuous batching 상황을 다룬다. 이 단계적 구성은 LLM inference engine 최적화의 핵심 요소를 Python 생태계 안에서 재현하고 비교할 수 있게 한다. 기존의 AI 기반 migration workflow 연구가 정적 분석, migration plan, orchestrator/coder 협업, 품질 평가를 결합해 대규모 모델 이전을 지원한 것처럼 [8], 본 연구는 LLM serving 커널을 cuTile/CUDA Python 기반으로 재구성하면서 각 단계의 설계 선택과 성능 결과를 함께 기록한다.

2.3 시스템 최적화 파트너로서의 코딩 에이전트

최근 코딩 에이전트는 코드 생성, 테스트 수정, 자동 디버깅을 넘어 시스템 최적화 과정에도 활용되고 있으며, GPU 커널 성능

최적화에 특화된 멀티 에이전트 시스템으로 Astra [7]가 제안되었다. 그러나 GPU 커널 최적화에서는 단순히 코드를 생성하거나 파라미터를 탐색하는 것만으로 충분하지 않다. 레지스터 스퀴어링, shared memory 사용량, L2 cache locality, kernel launch 비용, PyTorch allocator 동작, KV cache 갱신 방식처럼 서로 다른 계층의 원인이 하나의 성능 결과로 합쳐지기 때문이다. 따라서 에이전트가 유효한 도움을 주기 위해서는 현재 시스템의 구조와 실험 이력을 일관된 멘탈 모델로 유지해야 한다.

본 연구에서는 코딩 에이전트를 자율 실행 주체가 아니라 TechLead 역할의 실험 설계자와 분석자로 제한하였다. TechLead는 직접 실험을 실행하지 않고, DSL에 기록된 하드웨어 제약, 튜닝 파라미터, 이전 실험 결과를 바탕으로 패치와 측정 계획을 제안한다. 실제 실행은 Executor가 담당하고, 실행 로그와 측정 결과는 다시 DSL 지식으로 누적된다. 이러한 역할 분리는 에이전트가 로컬 디버깅의 우발적 세부사항에 끌려가지 않고, 시스템 수준의 인과관계와 아키텍처 제약을 중심으로 판단하도록 만드는 장치로 작동한다. 이는 Spec Kit Agents가 단계별 context-grounding hook과 validation hook을 통해 에이전트 산출물을 repository evidence에 연결하려는 접근 [9]과 유사하게, 실험 DSL을 통해 TechLead의 판단 근거를 명시적으로 고정한다는 점에서 관련된다.

3. 공동 설계 방법론

3.1 프레임워크 개요

단일 GPU의 제한된 하드웨어 자원 하에서 LLM 추론 엔진 커널을 마이그레이션하려면, 고수준 아키텍처 모델과 저수준 물리 연산 사이의 제약을 함께 관리해야 한다. 본 연구는 이 과정을 통제된 과학실험(Controlled Scientific Experiment)으로 모델링하고, 실험 결과를 DSL 지식으로 누적하여 다음 패치와 튜닝 제약을 갱신하는 인지 컴파일러(Cognitive Compiler) 방법론으로 정리한다.

3.2 3계층 위계 구조

본 프레임워크는 정보의 추상화 수준과 실행 권한에 따라 Architect(상위), TechLead(중위), Executor(하위)의 3계층으로 역할을 분리한다.

- **Architect**: 시스템 궁극 의도(Intent) 정의, 전역 제약 수립, 지식 최종 승인을 담당한다. 거시적 구조 설계와 핵심 기술 스택 선정에 관여하고, 이후 기술적 상세 수준의 전개에는 직접 개입하지 않는다.
- **TechLead-Agent**: 설계 문서, DSL 문서, 실험 로그, 실패 원인, 후속 제약을 연결하여 시스템 지식을 누적하고 정합성을

유지하는 중심 계층이다. Architect가 정의한 전역 의도와 제약을 구체적인 패치 후보, 튜닝 탐색 범위, acceptance criteria로 변환하고, Executor가 반환한 측정 결과를 다시 DSL 지식으로 구조화한다.

- **Executor:** 패치를 물리적으로 로드하여 GPU 상에서 JIT 컴파일 및 론칭, 실행 에러에 대한 로컬 자율 디버깅 수행, 프로파일 텔레메트리 계측을 수행한다. 전역적 아키텍처 설계에는 관여하지 않고 정량 데이터 수집에 집중한다.

3.3 DSL 기반 실험 지식 명세

본 연구의 DSL은 단순한 설정 파일이 아니라 실험 설계, 실행 결과, 실패 원인, 후속 제약을 누적하는 실험 지식의 저장 단위로 사용되었다. 각 단계의 DSL 문서는 공통적으로 metadata, design.space, tuning.space, hardware.constraints, knowledge.rules, milestones, agent.eval.loop, acceptance.criteria 필드를 가진다. 이 구조를 통해 TechLead는 이전 실험의 결과를 명시적인 규칙과 측정 기준으로 다시 읽고, 다음 패치와 실험 조건을 설계할 수 있었다.

DSL 필드	기록 내용	역할
design.space	Prefill/Decode, KV cache, execution mode	구조 선택지 정의
tuning.space	tile size, SM allocation, block size	튜닝 파라미터 정의
knowledge.rules	성공/실패에서 도출된 조건과 후속 action	insight 재사용
acceptance.criteria	throughput, TTFT, ITL, VRAM 등	결과 판정 기준

대표적으로 dynamic shape padding은 JIT compile 반복을 줄이기 위한 시도였지만, Eager-mode serving에서 매 decode step마다 대형 임시 tensor 할당과 복사를 유발하여 PyTorch caching allocator와 Python GC 비용을 크게 증가시켰다. 이 결과는 Allocator.Thrashing.Prevention 및 Caching.Allocator.Thrashing.Prevention 규칙으로 기록되었고, 후속 micro-vllm 실험에서는 no-padding Eager serving을 기본 제약으로 사용하였다.

4. micro-vllm 아키텍처 및 핵심 최적화

4.1 Windows-native 서빙 구성

vLLM과 같은 프로덕션 서빙 프레임워크는 대규모 모델의 분산 연산을 위해 Linux 계열에서만 작동하는 NCCL 백엔드를 기본적으로 사용한다. Windows 네이티브 환경에서 분산 텐서 연산의 컴파일 호환성을 검토하기 위해, 우리는 PyTorch Distributed의 Gloo 백엔드를 다중 디바이스 분산 스택에 이식하였다. 이를 통해 분산 통신 계층의 대대적인 코드 수정이나 이진 컴파일(Binary compilation) 충돌 없이, 텐서 병렬 선형 레이어의 논리

구조를 Windows 네이티브 실행 환경으로 비교적 적은 수정으로 마이그레이션할 수 있음을 확인하였다.

4.2 벡터화된 논리-물리 블록 캐시 재구성

실제 서비스 루프 내에서 prefix-cached prefill 연산을 수행할 때, 새롭게 인가된 Key, Value 텐서 모양과 메모리 풀 내부의 논리적 시퀀스 크기 간의 불일치로 인한 런타임 셰이프 오류가 빈번히 유발된다. 우리는 이 셰이프 매칭을 CPU 연산 루프를 사용하지 않고 GPU 상에서 병렬적으로 처리하기 위해, 벡터화된 논리-물리 캐시 재구성 레이어를 적용하였다. context 길이 seqlen.k에 해당하는 메모리 오프셋 주소를 텐서 인덱스 연산으로 변환하여 물리 주소로 매핑하며, dynamic continuous batching 상황에서의 형상 불일치 문제를 본 실험 범위에서 완화하였다.

4.3 FMHA 및 호스트-디바이스 병목 분석

FMHA(Fused Multi-Head Attention) 커널 설계 시, 메모리 계층 간의 IO 비용을 축소하기 위해 attention score를 공유 메모리 및 레지스터 영역에 상주시켰다. online softmax 알고리즘을 타일 레벨에 주입하여 수치 안정성을 취하고, causal masking을 cuTile 스트레드-블록 연산에 통합하였다.

반면 JIT 컴파일을 줄이기 위한 dynamic padding은 예상과 달리 성능을 저하시켰다. padding을 위해 매 decode step마다 대형 패딩 텐서를 새로 할당하면, PyTorch의 CUDA caching allocator가 내부 free-list 탐색과 cudaMalloc/cudaFree fallback, Python GC 비용을 반복적으로 유발한다. 본 구현 범위에서는 no-padding Eager serving이 더 나은 선택이었다.

4.4 CUDA Green Context 기반 자원 격리

우리는 CUDA Python 1.0의 하드웨어 분할 API인 Green Contexts를 적용하여 RTX 5070 GPU의 전체 48개 SM 자원을 Prefill 32 SM, Decode 16 SM으로 분할하였다. 이러한 물리적 SM 구획화는 Prefill과 Decode의 실행 영역을 분리하고 Decode 작업의 L2 cache residency를 보호할 가능성이 있다. 다만 현재 구현은 Prefill과 Decode를 서로 다른 CPU thread와 GPU stream에서 overlap 시키지 못하므로, 안정적인 E2E 성능 개선은 추가 검증 대상으로 남는다.

5. 실험 및 평가

5.1 실험 환경

Windows 11 OS 환경에서 NVIDIA GeForce RTX 5070 GPU(48 SM, 12GB VRAM, CUDA 13.3) 및 Qwen2.5-3B-Instruct 모델을 사용해 평가를 실시하였다. 동시 사용자 수는 256명으로 고정하고, 입력 및 출력의 크기를 100에서 1024 토큰 범위 내에서 동적으로 조율하는 continuous batching 상황을 재현하였다.

5.2 WSL FlashAttention 기준선과 cuTile 비교

동일하게 133,966개 생성 토큰을 처리하는 dynamic multi-user benchmark에서 Windows native cuTile backend와 WSL2 FlashAttention backend를 비교하였다. FlashAttention은 Linux/WSL2 환경에서 Triton 및 optimized attention kernel을 활용하는 성숙한 기준선으로 사용하였으며, cuTile backend는 Windows native kernel migration 경로의 현재 구현 성능을 확인하는 대상으로 사용하였다.

Backend	Runs	Time	Throughput	Rel.
Win. cuTile	3	289.16s	463.35 tok/s	1.00x
WSL2 FlashAttn	4	62.65s	2138.55 tok/s	4.62x

반복 측정 결과 FlashAttention 기준선은 평균 2138.55 tok/s를 기록하여 현재 cuTile backend 평균 463.35 tok/s 대비 약 4.62배 높은 처리량을 보였다. 따라서 본 연구의 Windows native cuTile 구현은 production-grade Linux attention kernel을 능가한다는 주장이 아니라, Windows 환경에서 LLM serving kernel을 단계적으로 마이그레이션하고 병목을 분석할 수 있는 구현 및 학습 경로로 해석해야 한다.

5.3 Allocator thrashing 및 Green Contexts 결과

dynamic padding 설정은 Eager 설정에 비해 전체 시스템 Throughput이 67.0% 하락하는 병목을 유발하는 것으로 관찰되었다. 이는 launch/JIT 회피 전략도 메모리 할당 패턴과 KV cache 갱신 방식까지 함께 고려해 평가해야 함을 보여준다.

Green Contexts 반복 측정에서는 초기 단일 실행에서 Decode P50 ITL과 throughput 개선이 관찰되었지만, 추가 반복 실행을 포함한 총 8회 측정에서는 일관된 throughput 개선으로 재현되지 않았다.

Metric	Green OFF	Green ON	Delta
TTFT	251.79 ms	250.30 ms	-0.6%
Decode P50 ITL	49.76 ms	52.33 ms	+5.2%
Decode P99 ITL	91.14 ms	87.60 ms	-3.9%
Throughput	397.07 tok/s	386.92 tok/s	-2.6%

8회 반복 결과는 Green Contexts 기반 SM 분할이 단일 GPU 내부에서 Prefill과 Decode의 실행 영역을 물리적으로 나누는 데에는 유효하지만, 현재의 순차적 Python engine loop에서는 안정적인 처리량 개선으로 직접 이어지지 않는다는 점을 보여준다. 다만 Decode P99 ITL은 평균 3.9% 감소하여 tail latency 완화 가능성이 관찰되었다.

6. 결론 및 향후 연구

본 논문은 Windows OS 환경에서 cuTile DSL과 CUDA Python 1.0을 결합한 cuTile 기반 LLM 추론 엔진인 micro-vllm을 구현하고 E2E 수준의 병목 요인을 관찰하였다. WSL2 FlashAttention 기

준선은 현재 Windows native cuTile backend보다 평균 4.62배 높은 처리량을 보였으므로, micro-vllm의 현재 기여는 production-grade kernel 대비 성능 우위가 아니라 Windows native migration, 교육 가능한 실험 경로, 그리고 병목 분석 프레임워크에 있다. Green Contexts 기반 SM 파티셔닝은 총 8회 반복 실행에서 Decode P99 ITL을 평균 3.9% 낮추었지만, P50 ITL과 throughput은 각각 5.2%, 2.6% 악화되었다. 따라서 본 논문은 이를 단일 GPU 내 Prefill/Decode 자원 격리의 구현 가능성과 tail-latency trade-off에 대한 초기 실증으로 해석하며, 안정적인 성능 개선 주장은 추가 검증 대상으로 남긴다.

향후 연구로는 엔진 루프를 비동기 멀티스레드 구조로 확장하여, 서로 다른 GPU stream과 Green Contexts 기반 SM 파티션에서 Prefill과 Decode가 동시에 진행되도록 할 계획이다. 또한 반복되는 system prompt나 RAG prefix에 대해 Prefix KV cache reuse를 적용하여 Prefill 연산을 줄이고, 더 긴 context와 높은 동시성을 지원하기 위한 선택적 KV cache offloading 기법을 후속적으로 검토한다.

자료 공개

본 논문에 사용된 구현 코드와 주요 측정 결과는 다음 공개 저장소에서 확인할 수 있다: <https://github.com/taewony/metaprogramming/tree/main/KernelAgent>.

감사의 글

본 연구의 코드 최적화 및 실험 설계 과정에서 AI 코딩 에이전트를 보조 도구로 활용하였으며, 모든 최종 결과와 검증은 인간 저자가 수행하였다.

참고 문헌

[1] W. Kwon, Z. Li, S. Zhuang, et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), 2023.

[2] NVIDIA Corporation, “NVIDIA CUDA 13.3 Enhances GPU Development with Tile Programming in C++ Compiler, Auto-tuning, and Python Updates,” NVIDIA Developer Blog, 2026.

[3] P. Tillet, H. T. Kung, and D. Cox, “Triton: an intermediate language and compiler for tiled neural network computations,” Proceedings of MAPL, 2019.

- [4] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” NeurIPS, 2022.
- [5] PyTorch Foundation, “torch.cuda.green_contexts: Granular Resource Partitioning for CUDA Kernels,” PyTorch Beta Documentation, 2025.
- [6] NVIDIA Corporation, “cuTile Python API Reference Guide,” NVIDIA CUDA Documentation, 2026.
- [7] A. Wei, T. Sun, Y. Seenichamy, H. Song, A. Ouyang, A. Mirhoseini, K. Wang, and A. Aiken, “Astra: A Multi-Agent System for GPU Kernel Performance Optimization,” arXiv preprint arXiv:2509.07506, 2025.
- [8] S. Nikolov, B. Konrad, M. Gronbach, N. Kumar, A. Yan, V. Singh, Y. Liang, and P. Ranganathan, “A Multi-agent AI System for Deep Learning Model Migration from TensorFlow to JAX,” arXiv:2603.27296, 2026.
- [9] P. Taghavi and S. Bhavani, “Spec Kit Agents: Context-Grounded Agentic Workflows,” arXiv:2604.05278, 2026.